Atelier 4 -Tester, sécuriser et appeler l'API LogiAI

LogiAI reste l'exemple. Dans la séance précédente, le modèle est devenu une API. Aujourd'hui, vous essayez de la casser avec des tests, vous la protégez par une clé, puis une page web l'appelle.

Ce que vous produisez : Atelier 4 avec son COMPTE_RENDU.md rempli, rendu par une pull request et tagué tp4.

Étape	Ce que vous faites
1. Préparer	dépôt, installation, premier constat : l'API est ouverte
2. Tester	casser son API : cinq requêtes, cinq tests, deux bugs à trouver
3. Qualité	ruff : inspecter son code sans l'exécuter
4. Sécuriser	la clé API, le .env, CORS (cours court)
5. TP, sprint 1	la clé protège /v1, les tests de sécurité passent
6. TP, sprint 2	une page web appelle l'API ; expérience CORS
7. Rendre	ADR 003, compte rendu, pull request, tag tp2, ticket de sortie

1 · Préparer (15 min)

En binôme. Un membre crée la branche, l'autre la récupère :

```
# --- un membre ---
git switch main
git pull
git switch -c tp-securite
# copier le contenu de logiai_starter.zip dans atelier4
git add TP
git commit -m "Atelier4 : starter"
git push -u origin tp-securite

# --- chaque membre ---
git pull
git switch tp-securite
cd atelier4
python -m venv .venv
.venv\Scripts\activate.bat
pip install -r requirements.txt
python -m logiai.dataset
python -m logiai.train
python -m pytest
```

> Pourquoi un nouvel environnement ? Celui du TP précédent n'a pas python-dotenv. Les 10 échecs sont NORMAUX : ce sont les tests de sécurité que vous écrirez, et le test d'exemple qui attend un 401 alors que l'API répond encore 200.

Premier constat : l'API est ouverte à tous

Lancez l'API (python -m uvicorn logiai.api.main:app --reload), puis dans un autre terminal :

```
curl.exe -X POST http://localhost:8000/v1/predict \
  -H "Content-Type: application/json" \
  -d "@livraison.json"
```

livraison.json contient une livraison valide : distance 120, poids 450, heure 17, pluie 8, trafic 7.5, 4 arrêts, camion. Aucune clé, et pourtant la réponse est 200. C'est le problème du jour.

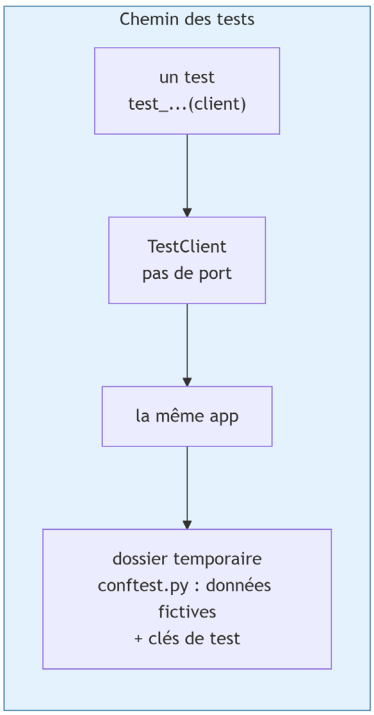
> **À montrer : la branche tp-securite sur GitHub et le 200 obtenu sans clé.**

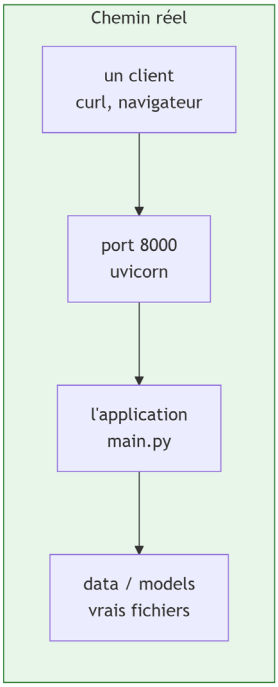
2 · Tester une API

Un test est une question précise avec la réponse attendue : « si j'envoie une pluie de 500 mm, l'API répond-elle 422 ? ». Jusqu'ici les tests étaient fournis ; aujourd'hui vous en écrivez.

TestClient : tester sans serveur.

Dans les tests, on ne lance pas uvicorn. TestClient envoie les requêtes directement à l'application, sans port. Les fixtures de tests/conftest.py fournissent un client, une livraison valide et des clés de test : les tests n'utilisent jamais vos vraies clés.





Analogie LogiAI : *le simulateur de réception. Avant d'ouvrir un entrepôt, on teste la procédure de réception avec un camion fictif, sans toucher aux vrais stocks. Les tests font la même chose : un client fictif, des clés fictives, la même application.*

Anatomie d'un test

```
VALID = {"distance_km": 120.0, "poids_kg": 450.0, "heure_depart": 17,
         "pluie_mm": 8.0, "trafic_index": 7.5, "nb_arrets": 4,
         "type_vehicule": "camion"}

def test_pluie_de_500_mm_est_refusee(client, headers_user):
    # client et headers_user : des fixtures de conftest.py
    r = client.post("/v1/predict", json={**VALID, "pluie_mm": 500},
                    headers=headers_user)
    assert r.status_code == 422   # la réponse attendue
```

Ce que chaque morceau veut dire

Morceau	Sens
def test_...(client)	pytest lance toute fonction dont le nom commence par test_ et lui donne la fixture client
{VALID, "pluie_mm": 500}**	une copie de VALID où une seule valeur change
client.post(url, json=...)	la requête, comme /docs ou curl.exe l'enverraient
assert ... == 422	si c'est faux, le test échoue et pytest affiche les deux valeurs (obtenue, attendue)

Ce qu'on peut tester dans LogiAI

Ce qu'on teste	Comment	Exemple
les données	pytest et logiai.dataset	les colonnes attendues du CSV
le modèle	entraînement temporaire	il bat la baseline
le cas nominal	TestClient	une livraison normale donne 200
les entrées refusées	TestClient	une pluie de 500 mm donne 422
la sécurité	TestClient et un en-tête	sans clé, 401
le navigateur	une requête OPTIONS	une origine inconnue est refusée

Activité 1 · Casser son API (25 min)

En binôme.

1. Écrivez cinq requêtes qui devraient être refusées, et prévoyez le code de chacune AVANT de l'envoyer :

Cas	Requête (corps ou URL)	Prévu	Obtenu
champ manquant			
valeur absurde			
texte au lieu d'un nombre			
champ en trop			
mauvaise route			

2. Lancez python -m uvicorn logiai.api.main:app --reload, envoyez-les depuis /docs, remplissez « Obtenu ». Une prévision fausse est un bon résultat : elle montre ce que vous aviez mal compris.

3. Transformez-les en tests dans tests/test_casser.py. Un seul test, cinq cas, grâce à parametrize : pytest lance la fonction une fois par ligne de la liste et nomme chaque cas dans le rapport. Le fichier contient un cas d'exemple et la fonction de test ; vous ajoutez les quatre autres.

```
CAS_REFUSES = [
    # (description, url, corps, code attendu)
    ("valeur absurde", "/v1/predict", {**VALID, "poids_kg": 99999}, 422),
    # ... vos quatre autres cas
]

@pytest.mark.parametrize("description, url, corps, attendu", CAS_REFUSES)
def test_request_refuse(client, description, url, corps, attendu, headers_user):
    assert client.post(url, json=corps, headers=headers_user).status_code == attendu
```

Pour un champ manquant : SANS_POIDS = {k: v for k, v in VALID.items() if k != "poids_kg"} (VALID sans une clé, déjà écrit dans le fichier). python -m pytest -v affiche le nom de chaque cas.

4. Une requête acceptée alors qu'elle devrait être refusée ? Corrigez les bornes dans schemas.py et gardez le test.

À montrer : python -m pytest tests/test_casser.py avec vos cinq cas verts.

Activité 2 · Chasser les deux bugs (20 min)

Le projet contient deux bugs volontaires dans les bornes de logiai/api/schemas.py. Une valeur impossible est acceptée : seuls des tests aux bornes les révèlent.

1. Dans tests/test_limites.py, la liste CAS_LIMITES contient déjà quatre cas pour distance_km. Ajoutez les deux côtés de chaque borne pour les six autres champs : juste dedans (doit passer, 200) et juste dehors (doit être refusé, 422). Ajoutez aussi un entier décimal (par exemple une heure de 11,5).

2. Lancez python -m pytest tests/test_limites.py. Lisez chaque échec : le champ, la valeur, le code attendu, le code obtenu.

3. Une requête acceptée alors qu'elle devrait être refusée ? Corrigez les bornes dans schemas.py et gardez le test.

Indice : avez-vous testé les deux côtés de la borne de CHAQUE champ ? Si vous ne trouvez pas un bug, c'est que vos tests sont incomplets : c'est la leçon.

À montrer : python -m pytest tests/test_casser.py tests/test_limites.py tests/test_api.py tout vert, et vos deux bugs expliqués dans le compte rendu.

3 · La qualité du code : ruff (10 min)

pytest exécute le code et vérifie ce qu'il fait. **ruff** lit le code sans l'exécuter et signale ce qui est mal écrit : un import inutile (F401), des imports mal rangés (I001), une variable jamais utilisée.

Dans **atelier4**, environnement activé :

```
pip install ruff              # une seule fois
ruff check .                  # liste les problèmes ; [*] = corrigeable seul
ruff check . --fix            # corrige ce qui est sans risque
ruff format .                 # met le code en forme
python -m pytest              # vérifie que rien n'est cassé
```

Le code d'un atelier4 bien écrit donne : All checks passed!

Règle : ruff check . avant chaque commit, python -m pytest avant chaque push.

> **Attention dansatelier4 : le starter importe déjà les outils dont vous aurez besoin (Depends, secrets...). Tant que le code n'est pas écrit, ruff les signale "imported but unused" (F401), et ruff check . --fix les supprimerait. Ne lancez JAMAIS --fix dans atelier4 avant d'avoir fini le sprint 1.**

> **Analogie LogiAI :** *le contrôle qualité avant expédition. Un colis bien fermé, bien étiqueté, sans adhésif qui dépasse. Ruff ne regarde pas si le contenu est bon (pytest s'en charge) : il vérifie que le carton est propre.*

4 · Sécuriser une API (cours court)

Question	Réponse dans LogiAI
Qui appelle ?	authentification : une clé API dans l'en-tête X-API-Key ; sinon 401
A-t-il le droit ?	autorisation : une clé utilisateur sur une route réservée à l'administrateur ; sinon 403
Où sont les secrets ?	dans .env, lu au démarrage ; jamais dans le code ni dans Git
Depuis quel site ?	CORS : la liste des origines autorisées

L'ordre des contrôles

Une requête passe une suite de contrôles ; le premier qui échoue donne la réponse.

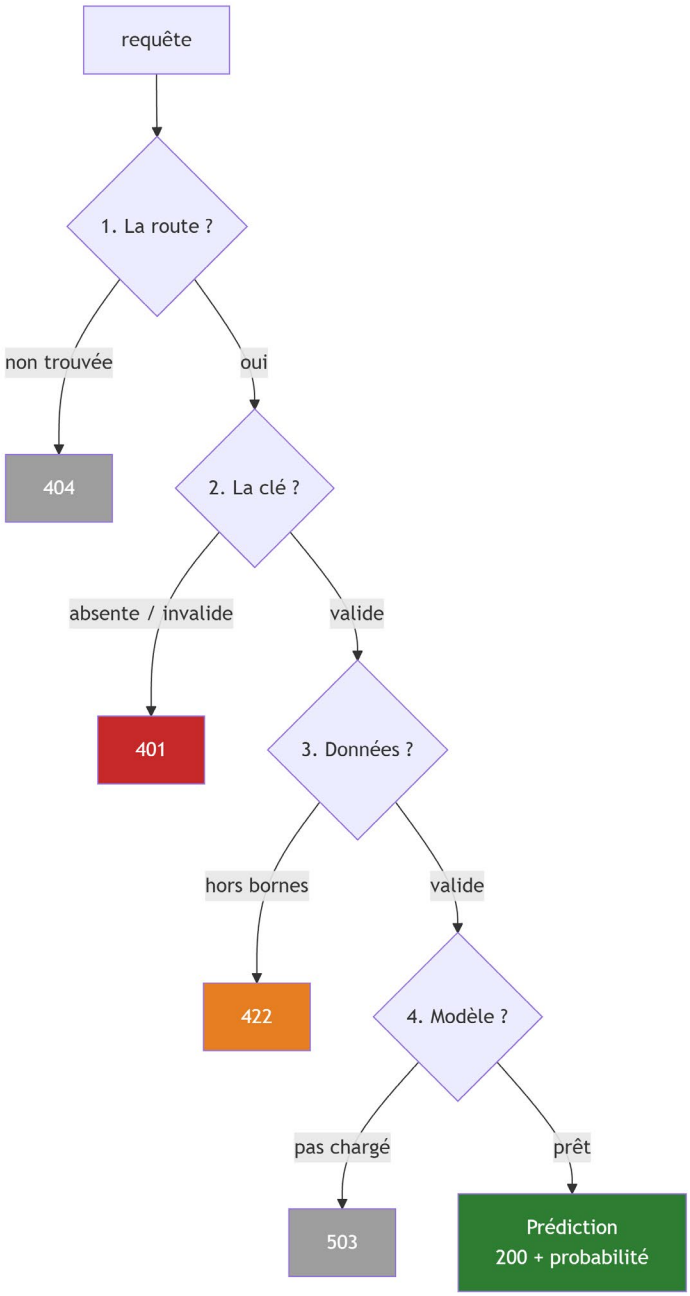


Schéma B — Ordre des contrôles (route → clé → données → modèle)

Analogie LogiAI : *la chaîne de réception d'un entrepôt. Le portail (la route existe-t-elle ?), le badge (la clé), le bon de livraison (le schéma), le quai (le modèle). Le premier contrôle qui échoue arrête le camion. Sans badge, on ne regarde même pas le bon de livraison : une pluie de 500 mm sans clé donne 401, pas 422.*

La clé voyage dans un en-tête, jamais dans l'URL

Une URL est enregistrée partout (journaux du serveur, historique du navigateur). Avec curl.exe, la clé est un en-tête de plus :

```
curl.exe -i -X POST http://localhost:8000/v1/predict \
  -H "Content-Type: application/json" \
  -H "X-API-Key: ma-cle" \
  -d "@livraison.json"
```

-i affiche aussi la ligne de statut. Sans clé ou avec une fausse clé : HTTP/1.1 401 Unauthorized.

401 ou 403 ?

401 : je ne sais pas qui vous êtes (clé absente ou fausse). 403 : je sais qui vous êtes, mais ce n'est pas permis (clé utilisateur sur la route /v1/model-info).

Où mettre la clé ?

Endroit	Bonne idée ?	Pourquoi
dans le code	NON	elle finit sur GitHub, pour toujours
.env	Oui	lu au démarrage, ignoré par Git
.env.example	Oui	commité avec de fausses valeurs : il dit quoi configurer
une variable du serveur	Oui	en production

Une clé commitée une seule fois reste dans l'historique de Git, même si on supprime le fichier ensuite. Il faut alors la changer. Avant chaque commit : git status, et .env ne doit pas apparaître.

CORS : le navigateur demande la permission

Une origine, c'est un protocole, un domaine et un port : http://localhost:5500 et http://localhost:8000 sont deux origines différentes. Quand une page appelle une autre origine, le navigateur envoie d'abord une requête OPTIONS (« puis-je ? »). Si l'API ne répond pas Access-Control-Allow-Origin, le navigateur bloque.

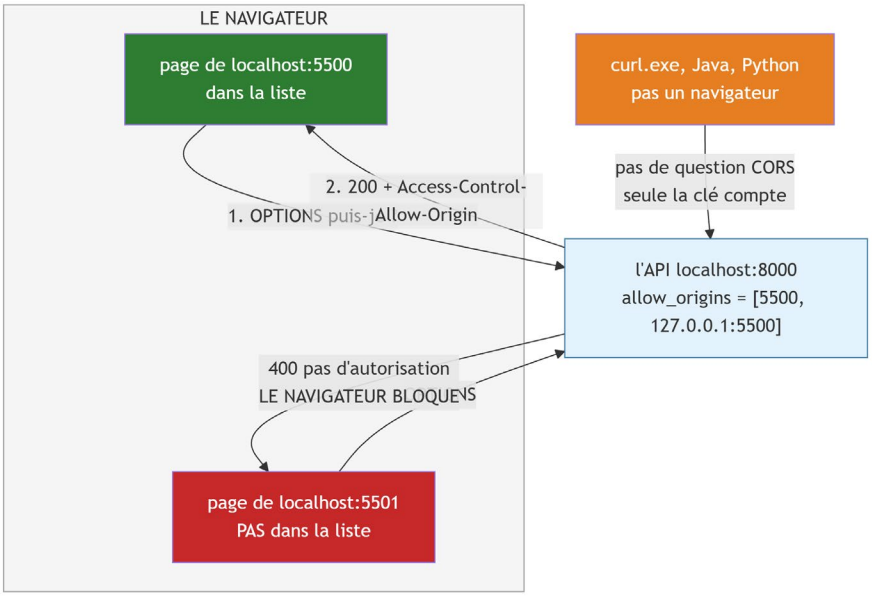


Schéma D — CORS : 5500 autorisé, 5501 bloqué, curl passe

Analogie LogiAI : *le gardien du hub. Le gardien (le navigateur) ne laisse pas un transporteur (la page) lire un colis d'un autre transporteur (l'API) sans que le hub ait dit « ok, tu es dans ma liste ».*

CORS est une règle DU NAVIGATEUR : il protège l'utilisateur contre une page qui utiliserait son navigateur pour appeler l'API. Ce n'est pas une protection de l'API : curl passe. La clé reste indispensable.

TP · Clé API, CORS et client web

Contexte.

L'API LogiAI ne doit répondre qu'aux programmes autorisés : chacun envoie une clé dans l'en-tête X-API-Key. La clé ne doit jamais apparaître dans le code ni dans Git. Une page web servie sur http://localhost:5500 doit pouvoir appeler l'API ; une page servie ailleurs, non.

Ce qui est fourni, ce que vous écrivez

Fichier	Rôle	À écrire ?
logiai/config.py	lit les clés et les origines	Fourni
logiai/api/security.py	verify_api_key() et require_admin()	à écrire
logiai/api/main.py	routes, avec trois TODO (Depends 1, Depends 2, CORS)	à écrire
.env.example	les variables à configurer	à copier
web/index.html	la page, avec quatre TODO	à écrire
tests/	test_api, test_limites, test_casser, test_securite	à completer
docs/adr/003-securite.md	deux décisions à justifier	à écrire
COMPTE_RENDU.md	à remplir au fil du TP	à remplir

Sprint 1 · La clé API (40 min)

1. logiai/api/security.py : écrivez verify_api_key() et require_admin().

La première compare la clé reçue à config.ADMIN_KEY puis à config.API_KEY avec _egal(...), renvoie le rôle ("admin" ou "user"), et lève une 401 si la clé est absente ou fausse. La seconde lève une 403 si le rôle n'est pas "admin".

pytest ne change presque pas : la fonction existe, mais rien ne l'appelle encore.

2. logiai/api/main.py : TODO 1 et TODO 2. Ajoutez dependencies=[Depends(...)] à /v1/predict et à /v1/model-info. /health reste public.

3. Vos clés : copy .env.example .env (macOS/Linux : cp), ouvrez .env, choisissez deux clés différentes. Relancez uvicorn : --reload surveille le code, pas le .env.

4. /docs : un champ x-api-key apparaît. Essayez : sans clé (401), clé utilisateur sur /v1/predict (200), clé utilisateur sur /v1/model-info (403), clé administrateur (200).

5. tests/test_securite.py : écrivez les neuf tests marqués « À écrire », sur le modèle du test d'exemple.

À montrer : python -m pytest tout vert, un 401, un 403 et un 200 devant vous.

Pistes, sans la solution

À écrire	Pistes

lire la clé du client	x_api_key: str | None = Header(default=None) lit l'en-tête X-API-Key ; None s'il est absent
la clé attendue	config.API_KEY et config.ADMIN_KEY ; elles valent None si .env n'a pas été chargé
comparer	_egal(a, b) (fournie) utilise secrets.compare_digest : == s'arrête au premier caractère faux, et la durée de la réponse trahirait combien de caractères étaient justes
refuser	raise HTTPException(status_code=401, detail="...")
protéger une route	Depends(verify_api_key) : on donne la fonction sans parenthèses ; FastAPI l'appelle à chaque requête
tester sans clé	ne pas passer headers= dans client.post
clés non configurées	monkeypatch.setattr(config, "API_KEY", None) : l'API doit tout refuser (échec sûr)

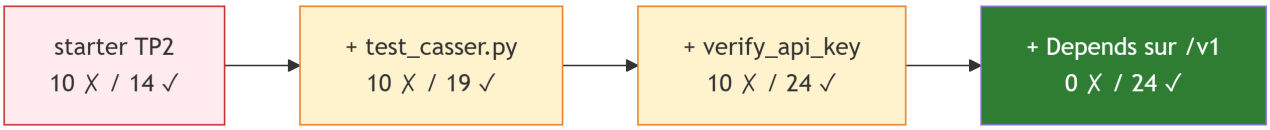


Schéma E — Progression des tests (starter → +test_casser → +security → +Depends)]

Sprint 2 · Le client web (40 min)

1. web/index.html, TODO 4 : dans la fonction appelerApi, envoyez la livraison en POST JSON à l'API avec les en-têtes Content-Type et X-API-Key. La forme d'un appel fetch :

```
const reponse = await fetch(API_URL, {
  method: "POST",
  headers: { "Content-Type": "application/json", "X-API-Key": cle },
  body: JSON.stringify(livraison),     // l'objet devient du texte JSON
});
const corps = await reponse.json();    // la réponse, en objet
return { status: reponse.status, corps };
```

2. Second terminal, dans TP\TP2, même environnement : python -m http.server 5500 -d web, puis ouvrez http://localhost:5500. Pas de double-clic sur le fichier : son origine serait null, refusée.

3. Saisissez votre clé et envoyez une livraison. Avant d'ajouter le CORS, ouvrez la console (F12) et notez le message d'erreur dans le compte rendu.

4. logiai/api/main.py, TODO 3 : ajoutez CORSMiddleware avec les origines de config.ALLOWED_ORIGINS, les méthodes GET, POST, OPTIONS et les en-têtes Content-Type et X-API-Key. Redémarrez uvicorn, rechargez la page.

5. Trois améliorations d'ergonomie (TODO 5, 6, 7) : un 422 lisible, le bouton désactivé pendant l'appel, un message clair quand l'API est injoignable. Notez ce qui s'affiche pour : une livraison normale, une pluie de 500 mm, une fausse clé.

6. Expérience CORS. Arrêtez le serveur de la page (Ctrl+C), relancez-le sur le port 5501, ouvrez http://localhost:5501. Que disent la page et la console (F12) ? Pourquoi curl.exe marche-t-il quand même ?

7. La clé est visible. Pendant un appel réussi : F12, onglet Network, la requête predict, en-têtes de la requête. Où voyez-vous la clé ? Que se passerait-il si elle était écrite dans le code JavaScript de la page ?

À montrer : la page qui affiche une probabilité avec alerte, l'erreur CORS du port 5501 et la clé dans F12.

D'autres clients : un quai multi-transporteurs

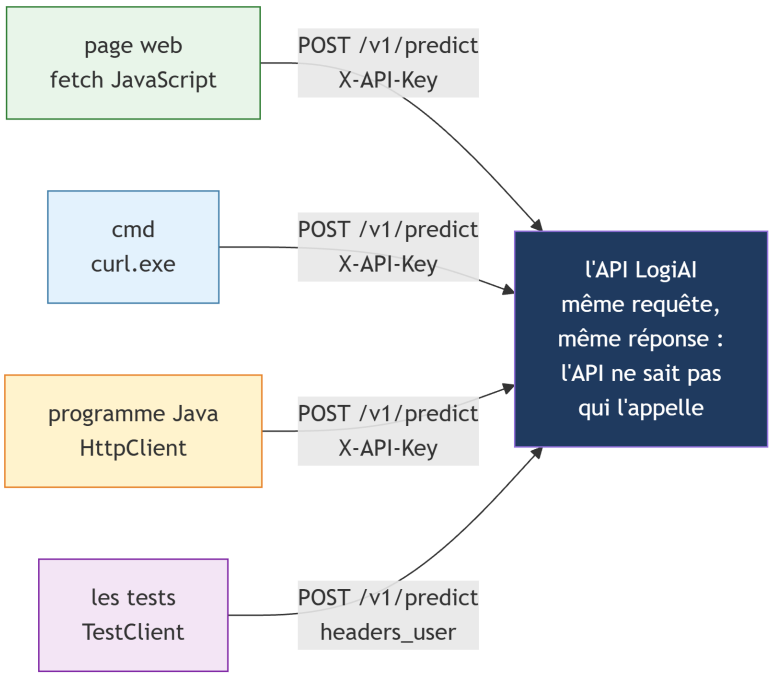


Schéma F — Multi-clients (page, curl, Java, tests) → même API]

Analogie LogiAI : *un quai normalisé. N'importe quel transporteur (page, curl, Java, test) qui respecte la fiche de déchargement (le contrat) peut se brancher au quai. Le quai ne sait pas qui arrive : il applique la même procédure.*

Les tests de l'activité 1, maintenant que l'API exige une clé

Votre test_casser.py passe déjà headers_user (la clé des tests). Pour comprendre l'ordre des contrôles : retirez temporairement headers=headers_user. Les cas 422 reçoivent 401, parce que la clé est vérifiée AVANT les données. Remettez-le.

Décision : l'ADR 003

Dans docs/adr/003-securite.md :

- où transmettre la clé (en-tête, URL ou corps ?)
- quelles origines autoriser (une liste précise ou * ?)

Pour chaque décision, écrivez ce que l'autre option ferait perdre.

Le compte rendu : COMPTE_RENDU.md

Le starter contient COMPTE_RENDU.md. Remplissez-le au fil de la séance : vos cinq requêtes de l'activité 1 (prévu, obtenu), vos deux bugs, votre verify_api_key(), les codes obtenus pour six requêtes, votre fetch, l'expérience CORS, cinq questions de compréhension et le lien de la pull request.

Rendre le TP (15 min)

```
python -m pytest              # tout vert
git status                    # .env ne doit PAS apparaître
git add .
git commit -m "TP sécurité : tests, clé API, page web"
git push
# GitHub : Compare & pull request ; un coéquipier relit, commente, approuve
# Merge pull request (Create a merge commit)
git switch main
git pull
git tag tp2
git push --tags
```

Le relecteur vérifie : pytest passe chez lui ; aucun fichier .env dans les changements ; l'ADR 003 et COMPTE_RENDU.md sont remplis ; au moins un commentaire.

Critères de réussite

- tous les tests passent ;
- sans clé, 401 ;
- clé utilisateur sur /v1/model-info, 403 ;
- /health reste public ;
- la page prédit depuis le port 5500 et échoue depuis 5501 ;
- .env n'est pas dans Git ;
- la pull request est fusionnée et le tag tp2 existe.

En cas de problème

Symptôme	Solution
ModuleNotFoundError: logiai	lancez les commandes depuis TP\TP2
uvicorn introuvable	activez l'environnement, ou utilisez python -m uvicorn

toutes les requêtes : 401, même avec la bonne clé	.env absent ou mal nommé, ou serveur non redémarré après modification
/health indique auth_configured: false	les deux variables du .env ne sont pas définies
Depends(verify_api_key()) : erreur au lancement	passez la fonction, sans parenthèses
403 attendu, 401 reçu	route testée sans en-tête, ou clé invalide
un test de limite reste rouge	c'est peut-être un des 2 bugs : lisez le message attendu / obtenu
.env apparaît dans git status	.gitignore modifié ; .env déjà poussé : git rm --cached .env, commit, push, et changez les clés
la page affiche « API injoignable »	uvicorn arrêté, mauvais port, ou CORS (console F12)
« blocked by CORS policy »	CORSMiddleware absent, ou allow_headers sans X-API-Key
la page ouverte par double-clic ne marche pas	servez-la : python -m http.server 5500 -d web
nouvelle origine ajoutée, toujours refusée	redémarrez uvicorn : la liste est lue au démarrage
curl : 422 JSON decode error	mettez le JSON dans un fichier : -d "@livraison.json"
ruff introuvable	pip install ruff dans l'environnement activé
NameError: name 'Depends' (ou secrets)	ruff --fix a supprimé l'import : remettez la ligne d'import du starter

Ticket de sortie (individuel, sur papier)

Nom : _____ Groupe : _____

1. Quelle différence entre 401 et 422 ?

2. Pourquoi la clé n'est-elle ni dans le code ni dans Git ?

3. Pourquoi curl fonctionne-t-il alors que la page du port 5501 est bloquée ?

4. Qu'est-ce qui reste flou pour vous ?